SHADOWFOX | MACHINE LEARNING ENGINEER INTERNSHIP

Beginner Level Project Report

Boston House Price Prediction

Predicting median home values from housing and neighbourhood features with five regression models

Submitted by	Samarth Jain
Institute	Sage University, Indore
Internship role	Machine Learning Engineer Intern
Organization	ShadowFox (Virtual Internship Program)
Program ID	SF-PID-2026-W7613
Internship period	1 November 2026 – 30 November 2026
Project level	Beginner
Report date	November 2026

Contents

Abstract
1. Introduction
2. Problem Statement
3. Project Objectives
4. Scope of the Project
5. Dataset Description
6. Tools and Technologies
7. Methodology and Workflow
8. Data Preprocessing
9. Exploratory Data Analysis
10. Model Development and Implementation
11. Model Evaluation and Results
12. Supplementary Check: Imputation Fitted on Training Data Only
13. Challenges Faced and Solutions
14. Results and Discussion
15. Limitations
16. Future Improvements
17. Conclusion
18. Reproducibility
References

Abstract

This report documents a supervised machine-learning project from the ShadowFox Machine Learning Engineer Internship that predicts median house values (MEDV) in the Boston area from 13 housing, neighbourhood and socioeconomic features. The dataset contains 506 records, of which 120 values (20 in each of six columns) were missing. Missing values were filled with the column median (the mode for the binary CHAS feature), the data was split 80/20 into 404 training and 102 test records, and five regression models were trained and compared: Linear Regression, Decision Tree, Gradient Boosting, Random Forest and Support Vector Regression (SVR).

On the held-out test set, Random Forest achieved the lowest MSE (9.1504) and the highest R^2 (0.8752), with an MAE of 2.2140, and was selected as the best overall model. SVR achieved the lowest MAE (2.0810), and Linear Regression was the weakest baseline (R^2 0.6591). The exploratory analysis showed that the average number of rooms (RM) is positively related to price ($r = 0.695$), while the percentage of lower-status population (LSTAT, $r = -0.723$) and the pupil-teacher ratio (PTRATIO, $r = -0.508$) are negatively related. The report also documents that imputation was performed before the train/test split, and a supplementary check shows this changed the test metrics only marginally.

1. Introduction

House prices depend on the characteristics of the home and of its neighbourhood: size, pollution, crime, access to services, school quality and the socioeconomic profile of the area. Regression models can learn these relationships from historical data and estimate the value of a house or neighbourhood from its features.

This project uses the classic Boston Housing dataset to follow a complete beginner-level machine-learning workflow: loading and inspecting the data, checking quality issues, handling missing values, exploring relationships, preparing training and test sets, training several regression models and comparing them with standard metrics.

2. Problem Statement

Given 13 attributes of a Boston-area tract (crime rate, zoning, business acreage, river proximity, air pollution, number of rooms, building age, distance to employment centres, highway access, tax rate, pupil-teacher ratio, a demographic-derived index and the percentage of lower-status population), predict the median value of owner-occupied homes, MEDV. Because MEDV is continuous, this is a supervised regression problem.

3. Project Objectives

- Understand and explore the dataset.
- Perform data cleaning and preprocessing.
- Analyse relationships between the features and house prices.
- Train multiple machine-learning regression models.
- Evaluate model performance with appropriate regression metrics (MAE, MSE and R^2).

- Compare the models and select the best overall performer.

4. Scope of the Project

The project covers one tabular dataset and a notebook-based workflow in Python: data inspection, missing-value handling, exploratory analysis, training five regression models with fixed hand-chosen settings, and comparison on a single held-out test set. It does not include systematic hyperparameter tuning, cross-validation, feature selection, model saving or a deployed application, so no application screenshots exist or are included.

5. Dataset Description

The file HousingData.csv contains 506 rows and 14 columns: 13 input features and the target MEDV. There are no duplicate rows. Table 1 summarises the columns.

Table 1: Dataset columns

Column	Description	Type
CRIM	Per capita crime rate	Float (20 missing)
ZN	Residential land zoned for large lots	Float (20 missing)
INDUS	Non-retail business acres	Float (20 missing)
CHAS	Charles River dummy variable (1 = tract borders the river)	Float, binary (20 missing)
NOX	Nitric oxide concentration	Float
RM	Average number of rooms	Float
AGE	Proportion of older owner-occupied units	Float (20 missing)
DIS	Weighted distance to employment centres	Float
RAD	Accessibility to radial highways	Integer
TAX	Property tax rate	Integer
PTRATIO	Pupil-teacher ratio	Float
B	Demographic-derived feature	Float
LSTAT	Lower-status population percentage	Float (20 missing)
MEDV	Median value of owner-occupied homes (target)	Float

The notebook does not state the unit of MEDV; in the original dataset documentation it is expressed in thousands of US dollars, and this report treats it as a relative price value, so all error metrics are in the same unit as MEDV.

5.1 Missing values

Six columns (CRIM, ZN, INDUS, CHAS, AGE and LSTAT) each have 20 missing values, which is 3.95% of the 506 rows per column and 120 missing values in total. The other eight columns are complete.

5.2 Target variable

Table 2: Summary statistics of the target variable MEDV

Statistic	MEDV
Mean	22.53
Median	21.20
Standard deviation	9.20
Minimum	5.00
Maximum	50.00
Houses at the maximum (50.00)	16

The 16 records at exactly 50.0 suggest that the target is capped at this value, which matters when interpreting predictions for expensive homes. The CHAS feature is highly imbalanced: 93.00% of the non-missing values are 0 and 7.00% are 1 (34 tracts border the river after imputation).

6. Tools and Technologies

Table 3: Tools and technologies

Tool	Purpose in this project
Python	Programming language for the whole workflow
Pandas	Data loading, inspection, cleaning and imputation
Matplotlib, Seaborn	Histogram, box plot, heatmaps, scatter plots and model-comparison charts
Scikit-learn	Train/test split, SimpleImputer, StandardScaler, Pipeline, the five regression models and the metrics
Jupyter Notebook	Interactive development environment
Git and GitHub	Version control and project hosting

7. Methodology and Workflow

The project follows the eight-step workflow in Figure 1, which mirrors the order of the notebook.

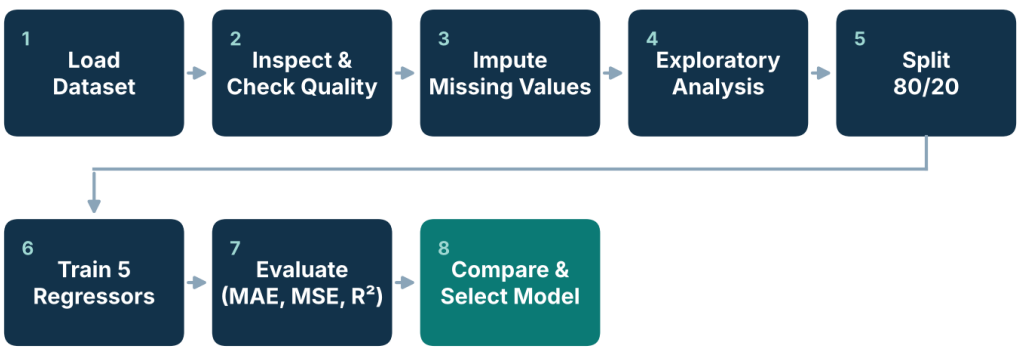


Figure 1: End-to-end workflow of the project (diagram prepared for this report from the notebook's structure)

8. Data Preprocessing

1. **Inspection.** Columns, size (7,084 values), shape (506, 14), data types and descriptive statistics were checked. All columns are numeric.
2. **Duplicates.** None were found (506 unique rows).
3. **Missing-value imputation.** Missing values in CRIM, ZN, INDUS, AGE and LSTAT were filled with each column's median, which is robust to extreme values; missing CHAS values were filled with the mode (0), which preserves the binary categories. A check confirmed zero missing values afterwards and an unchanged shape of (506, 14).
4. **Outlier screening.** The IQR rule on MEDV (Q1 = 17.025, Q3 = 25.0, IQR = 7.975) gives bounds of 5.06 and 36.96 and flags 40 potential outliers. They were inspected but kept, because unusual houses can be valid observations.
5. **Feature/target separation.** X contains the 13 predictors (506 × 13) and y contains MEDV.
6. **Train/test split.** The data was split 80/20 with random_state = 42, giving 404 training and 102 test records.

Important note on the order of operations. In the notebook, the median/mode imputation (step 3) was applied to the complete dataset before the train/test split. The later cells that fit SimpleImputer objects on the training data therefore had no missing values left to fill, and the final models were trained on the data imputed with full-dataset statistics. This can leak a small amount of test-set information into training. Section 12 quantifies the effect with an independent check.

SVR was trained inside a pipeline with StandardScaler. The other four models were trained on unscaled features, which is acceptable for tree-based models; Linear Regression does not require scaling to produce its predictions.

9. Exploratory Data Analysis

9.1 Target distribution and outliers

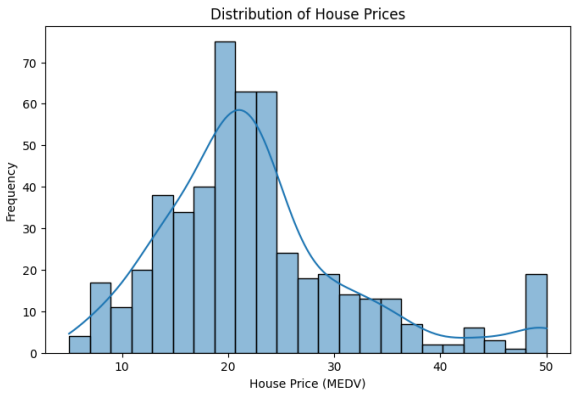


Figure 2: Distribution of house prices (MEDV)

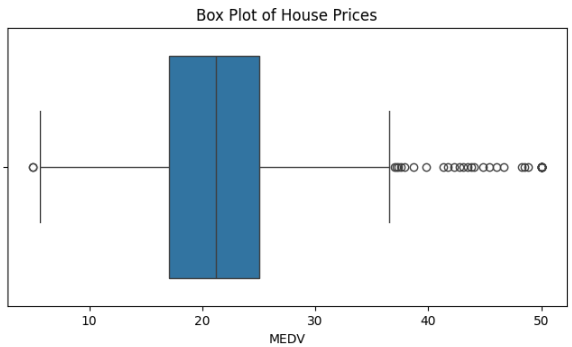


Figure 3: Box plot of house prices (MEDV)

MEDV is concentrated between about 17 and 25 with a median of 21.2 and a right-hand tail. There is a visible spike at 50.0 (16 houses), consistent with a capped target, and the box plot shows a cluster of high-value points above the upper whisker (36.96) and two low-value points at 5.0.

9.2 Correlation with MEDV

Table 4: Correlation of each feature with MEDV (notebook values, after imputation)

Negative: feature	r with MEDV	Positive: feature	r with MEDV
LSTAT	−0.723	RM	+0.695
PTRATIO	−0.508	ZN	+0.362
INDUS	−0.476	B	+0.333
TAX	−0.469	DIS	+0.250
NOX	−0.427	CHAS	+0.184
CRIM	−0.384		
RAD	−0.382		
AGE	−0.378		

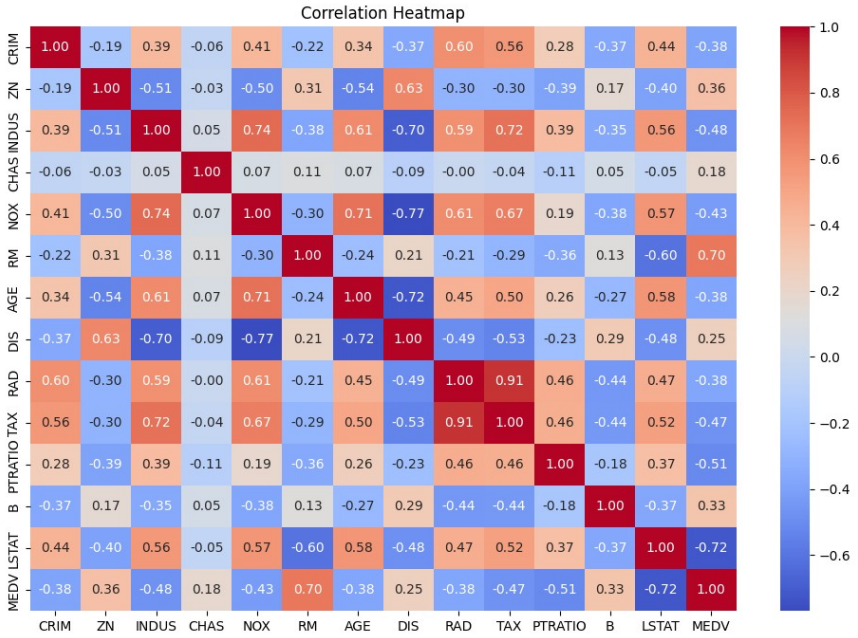


Figure 4: Correlation heatmap of all variables including MEDV

RM has the strongest positive correlation with MEDV (0.695): homes with more rooms are valued higher. LSTAT has the strongest negative correlation (−0.723), followed by PTRATIO (−0.508). Correlation measures linear association and does not establish causation.

9.3 Strongest relationships

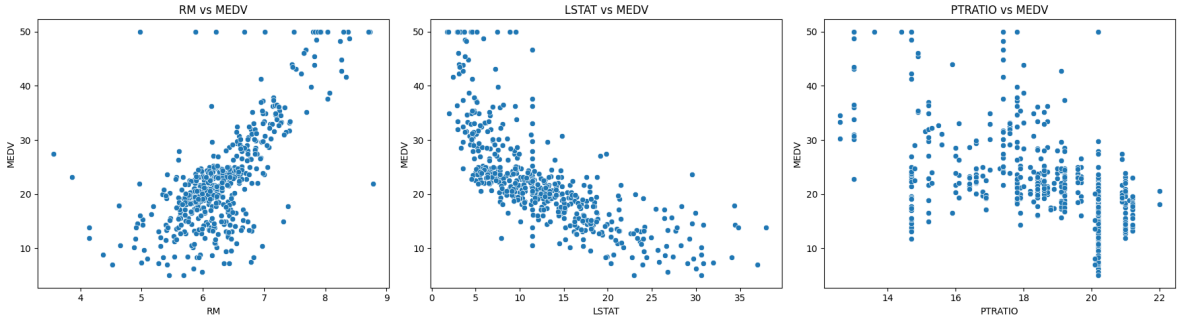


Figure 5: Scatter plots of MEDV against RM, LSTAT and PTRATIO

MEDV rises with RM, falls as LSTAT increases (a curved, non-linear decline) and tends to be lower at high PTRATIO values. The horizontal line of points at MEDV = 50 is visible in the RM and LSTAT plots. The curved LSTAT relationship helps explain why non-linear models later outperform Linear Regression.

9.4 Relationships among predictors

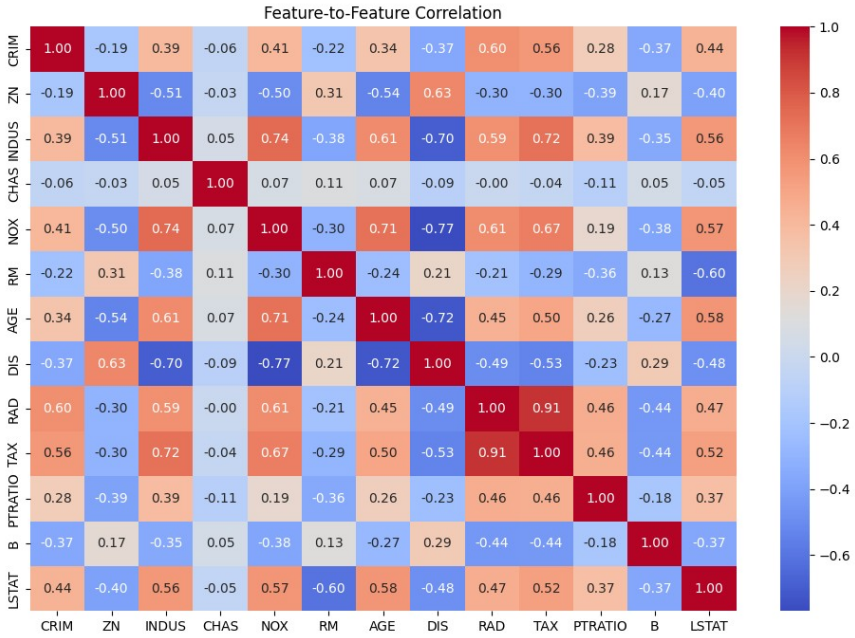


Figure 6: Feature-to-feature correlation heatmap (MEDV excluded)

Several predictors are strongly correlated with each other, most notably RAD and TAX (0.91), NOX and DIS (−0.77), INDUS and NOX (0.74), AGE and DIS (−0.72) and INDUS and TAX (0.72). This multicollinearity can make linear-model coefficients hard to interpret, while tree-based models handle overlapping information more flexibly.

10. Model Development and Implementation

Five regression models were trained on the 404 training records and evaluated on the 102 test records using MAE, MSE and R^2. The hyperparameters in Table 5 were set manually in the notebook; no systematic tuning or cross-validation was performed.

Table 5: Model configurations used in the notebook

Model	Configuration
Linear Regression	Default scikit-learn settings
Decision Tree Regressor	max_depth = 4, random_state = 42
Gradient Boosting Regressor	n_estimators = 100, learning_rate = 0.05, max_depth = 2, random_state = 42
Random Forest Regressor	n_estimators = 100, max_depth = 5, random_state = 42
Support Vector Regression	Pipeline of StandardScaler and SVR(kernel = “rbf”, C = 10, epsilon = 0.1)

MAE is the average absolute prediction error. MSE is the average squared error and penalises large mistakes more strongly. R^2 is the proportion of variance in MEDV explained by the model;

higher is better. RMSE, the square root of MSE, is also listed in Section 11 as a derived value in the original price units.

Linear Regression was trained first as an interpretable baseline. The fitted model has an intercept of 28.17, and its largest coefficients are NOX (−15.98), RM (+4.72), CHAS (+3.28), DIS (−1.51) and PTRATIO (−0.89). The coefficients should be read with care because the features are on different scales and are correlated with each other.

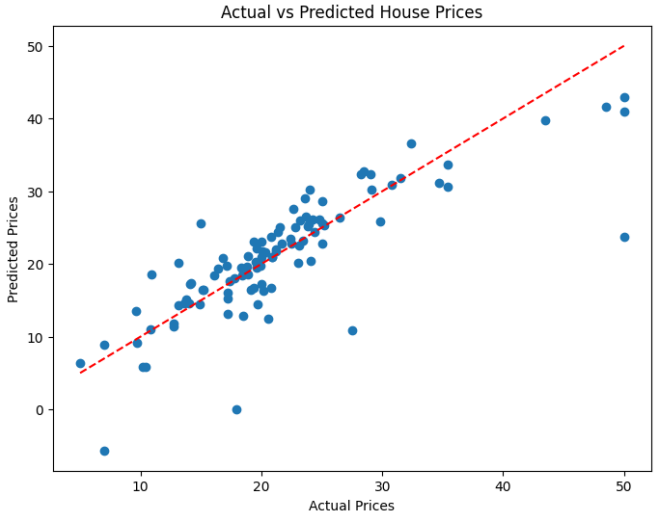


Figure 7: Linear Regression: actual versus predicted house prices on the test set (dashed line = perfect prediction)

Most predictions follow the diagonal, but there are visible misses: expensive homes at the 50.0 ceiling are under-predicted, and at least one prediction is negative, which is impossible for a price. This is a limitation of a linear model on a non-linear relationship and agrees with its low R^2 of 0.6591.

11. Model Evaluation and Results

Table 6: Test-set results (sorted by R^2); *RMSE is derived as √MSE

Model	MAE	MSE	RMSE*	R^2 Score
Random Forest	**2.2140**	**9.1504**	**3.0249**	**0.8752**
Decision Tree	2.7776	12.4201	3.5242	0.8306
Gradient Boosting	2.3624	12.4420	3.5273	0.8303
SVR	2.0810	13.9944	3.7409	0.8092
Linear Regression	3.1487	24.9994	4.9999	0.6591

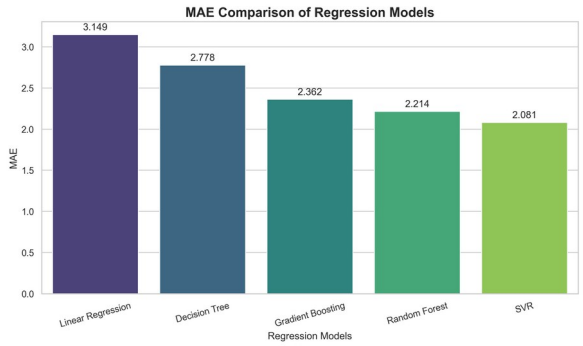


Figure 8: MAE comparison of the five models (lower is better)

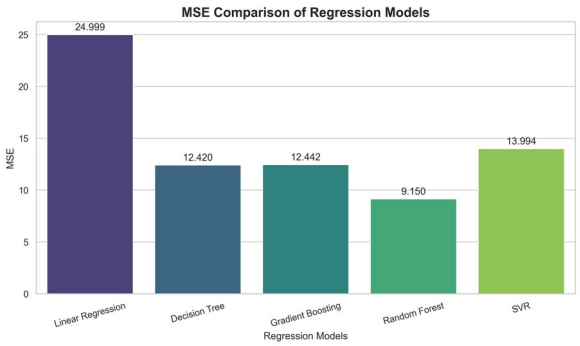


Figure 9: MSE comparison of the five models (lower is better)

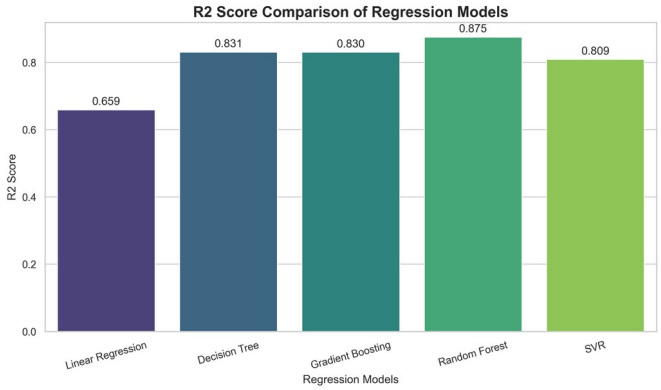


Figure 10: R^2 score comparison of the five models (higher is better)

Random Forest achieved the lowest MSE (9.1504) and the highest R^2 (0.8752), meaning it explains about 87.5% of the variance in MEDV on the test set and handles large errors best. SVR achieved the lowest MAE (2.0810), so its average absolute miss is the smallest, but its MSE (13.9944) is higher, which means it makes some larger errors. Decision Tree and Gradient Boosting are practically tied (R^2 0.8306 vs 0.8303), and Linear Regression is clearly the weakest (R^2 0.6591, MSE 24.9994).

Random Forest was selected as the best overall model because it leads on two of the three metrics and has a competitive MAE. The test set has only 102 records, so the differences between close models, such as Decision Tree and Gradient Boosting, should not be over-interpreted.

12. Supplementary Check: Imputation Fitted on Training Data Only

To measure the effect of imputing before the split (Section 8), the models were retrained for this report on the raw data with the same split (random_state = 42), fitting the median and mode imputers on the 404 training records only and applying them to the test records. All other settings were unchanged. This check was done during the preparation of this report and is not part of the original notebook.

Table 7: Effect of fitting the imputers on training data only

Model	R^2 (notebook → train-only)	MSE (notebook → train-only)	MAE (notebook → train-only)
Linear Regression	0.6591 → 0.6593	24.9994 → 24.9834	3.1487 → 3.1476
Decision Tree	0.8306 → 0.8306	12.4201 → 12.4201	2.7776 → 2.7776

Model	R^2 (notebook → train-only)	MSE (notebook → train-only)	MAE (notebook → train-only)
Gradient Boosting	0.8303 → 0.8304	12.4420 → 12.4347	2.3624 → 2.3641
Random Forest	0.8752 → 0.8755	9.1504 → 9.1302	2.2140 → 2.2171
SVR	0.8092 → 0.8096	13.9944 → 13.9619	2.0810 → 2.0709

The metrics change only in the third or fourth decimal place and the model ranking is unchanged, so the leakage was minor in this dataset (only about 4% of values per column were missing). Nevertheless, fitting preprocessing on the training data only is the correct practice and is recommended for future work.

13. Challenges Faced and Solutions

Table 8: Challenges and solutions

Challenge	How it was handled
Missing values in six columns (3.95% each)	Median imputation for numeric columns and mode imputation for the binary CHAS column; completeness was verified afterwards.
Possible outliers in MEDV (40 flagged by the IQR rule)	Inspected with RM, LSTAT and PTRATIO context and retained, instead of removing valid but unusual houses.
Capped target (16 houses at 50.0)	Identified during EDA and documented as a caution when interpreting predictions for expensive homes.
Scale-sensitive SVR	Trained inside a pipeline with StandardScaler so that scaling is learned from the training data only.
Correlated predictors (for example RAD and TAX, r = 0.91)	Identified with a feature-to-feature heatmap; tree-based models were included alongside Linear Regression.
Imputation applied before the split	Documented transparently and quantified in Section 12; the effect on the metrics is negligible.

14. Results and Discussion

All four non-linear models outperform Linear Regression by a wide margin (R^2 0.81–0.88 versus 0.66), which agrees with the curved LSTAT–MEDV relationship and the capped, skewed target seen in the EDA. Random Forest, which averages many shallow trees, gave the most accurate and stable predictions, with an RMSE of about 3.02 compared with a target standard deviation of 9.20.

The choice of the “best” model depends on the metric: SVR minimises the typical error (MAE), whereas Random Forest minimises the influence of large errors (MSE) and explains the most variance (R^2). For house-price estimation, where occasional large errors are costly, Random Forest is a reasonable overall choice. Because the hyperparameters were not tuned systematically and results come from one split, the comparison should be considered exploratory.

15. Limitations

- Hyperparameters were chosen manually and not tuned; no cross-validation was used, so the results rely on a single 102-record test set.

- Missing values were imputed using the complete dataset before the split; the supplementary check shows the effect is minimal, but it is not best practice.
- The target MEDV appears capped at 50.0 (16 records), so predictions for the most expensive homes are less reliable.
- The dataset is small (506 records) and describes Boston-area tracts from a past period; it may not represent current housing markets or other cities.
- The feature B is derived from the proportion of Black residents in a town. Because of this, the Boston Housing dataset has been removed from recent versions of scikit-learn for ethical reasons, and using such a feature in a real pricing system would raise serious fairness concerns. In this project the dataset is used only for educational model comparison.
- Some predictors are strongly correlated with each other (for example RAD and TAX), which limits the interpretation of linear-model coefficients.
- Linear Regression produced negative predicted prices for some test records.

16. Future Improvements

- Fit all preprocessing (imputation and scaling) inside a scikit-learn Pipeline so that it is learned from training data only.
- Use cross-validation and hyperparameter tuning (for example GridSearchCV or RandomizedSearchCV) instead of manually chosen settings.
- Add RMSE and cross-validated scores, and report feature importance for the tree-based models.
- Perform systematic feature selection, and evaluate the effect of removing sensitive features such as B.
- Try a log-transformed target or models that respect the price ceiling, and examine residuals for expensive homes.
- Test the approach on additional and more recent housing datasets.
- Save the final model and wrap it in a simple prediction interface.

17. Conclusion

This project built a complete beginner-level regression workflow on the Boston Housing data. After median/mode imputation of 120 missing values and an 80/20 split, five models were compared on the test set. Random Forest was selected as the best overall model (MAE 2.2140, MSE 9.1504, R^2 0.8752), with SVR achieving the lowest MAE (2.0810) and Linear Regression performing worst (R^2 0.6591). The EDA identified RM, LSTAT and PTRATIO as the most strongly related features. The main limitations are the single test split without tuning or cross-validation and the imputation before splitting, whose effect was shown to be negligible, so the next steps are to use pipelines, cross-validation and tuning.

18. Reproducibility

All metrics in Tables 6 and 7 were reproduced independently from HousingData.csv using the notebook's settings (random_state = 42, 80/20 split, the stated model hyperparameters) and match

the notebook's output. To rerun the project, install the dependencies (pip install pandas numpy matplotlib seaborn scikit-learn jupyter), place HousingData.csv in the same folder as the notebook and run the cells in order. The notebook does not save a trained model file.

References

[1] Harrison, D. and Rubinfeld, D. L. (1978). Hedonic housing prices and the demand for clean air. Journal of Environmental Economics and Management, 5(1), 81–102.

[2] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[3] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.

[4] Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. Annals of Statistics, 29(5), 1189–1232.

[5] Smola, A. J. and Schölkopf, B. (2004). A tutorial on support vector regression. Statistics and Computing, 14(3), 199–222.

[6] McKinney, W. (2010). Data structures for statistical computing in Python. Proceedings of the 9th Python in Science Conference, 56–61.

[7] Waskom, M. L. (2021). seaborn: statistical data visualization. Journal of Open Source Software, 6(60), 3021.

[8] Scikit-learn documentation: SimpleImputer, StandardScaler, RandomForestRegressor, GradientBoostingRegressor and SVR. https://scikit-learn.org/stable/